

UNIT-5

Classification and Prediction

Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian Classification
- Classification by back propagation
- Classification based on concepts from association rule mining
- Other Classification Methods-KNN
- Prediction
- Classifier Accuracy
- Summary

- **Databases are rich with hidden information** that can be used for making intelligent business decisions.
- Classification and Prediction are two forms of data analysis that can be used to extract models describing important data classes or to predict future data trends.
- ***Classification*** predicts categorical labels, ***prediction*** models continuous valued functions
- For ex, a classification model may be built to categorize bank loan applications as either safe or risky, while predication model may be built to predict the expenditures of potential customers on computer equipment given their income and occupation.
- Many classification and prediction methods have been proposed by researchers in machine learning, expert systems, statistics and neurobiology.

What is Classification ? What is Prediction

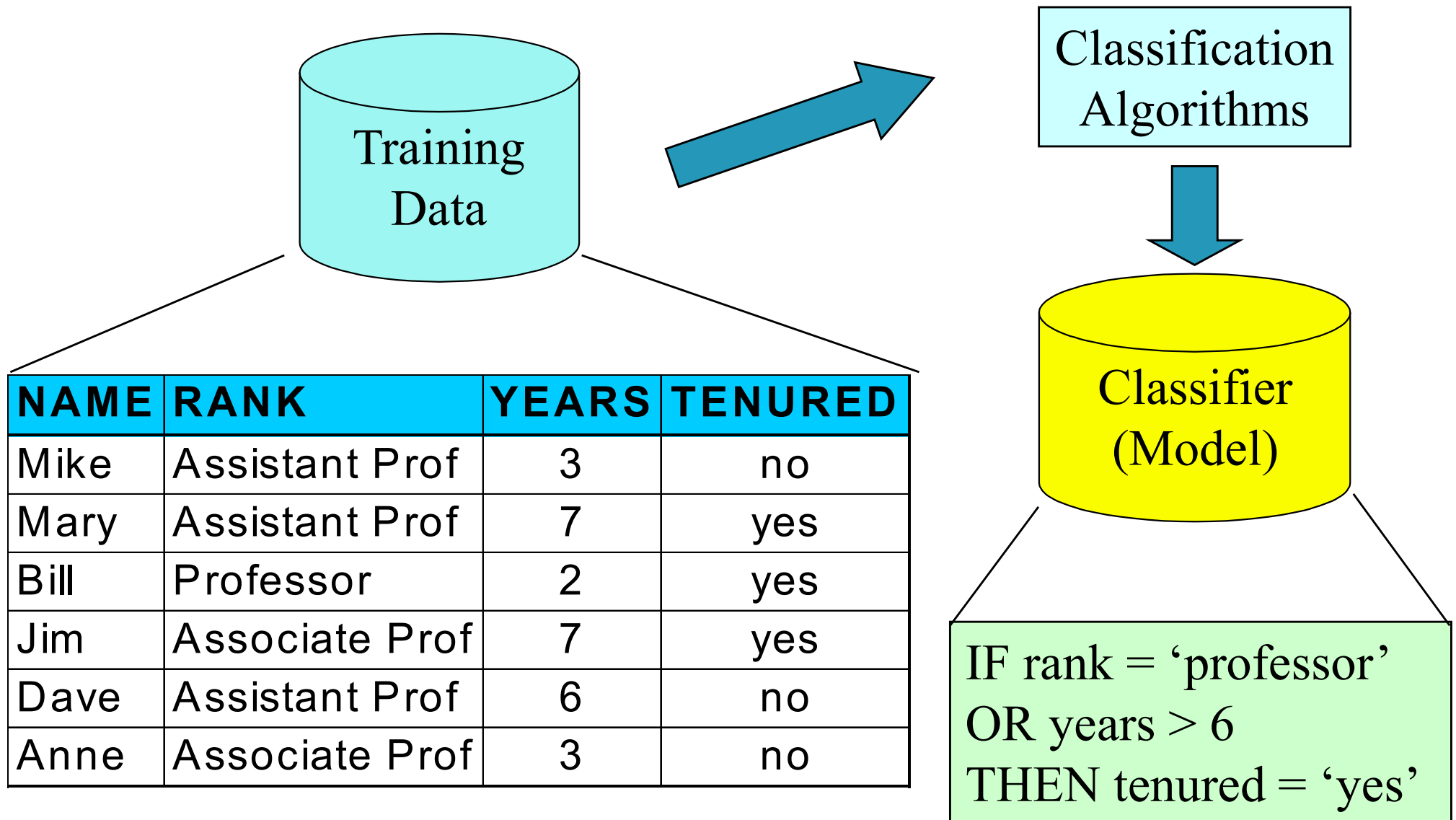
Data classification : It is a two step process.

- In the first step, a model is built describing a predetermined set of data classes or concepts- Learning or training phase, where a classification algorithm builds the classifier by analyzing or “learning from” a training set.
- Each Tuple is assumed to belong to a predefined class as determined by one of the attributes, called the **class label attribute**.

- In the context of classification, data tuples are also referred to as samples, examples or objects.
- The data tuples analyzed to build the model collectively form the **training data set**.
- The individual tuples making up the **training set** are referred to as **training tuples** and are randomly selected from the database under analysis.
- In the context of classification, data tuples can be referred as samples, examples, instances etc.,.

- Since the class label of each training sample is provided, this step is also known as **supervised learning** (i.e., the learning of the model is “supervised” in that it is told to which each training sample belongs)
- It contrasts with **unsupervised learning (or clustering)**, in which the class label of each training sample is not known, and the number or set of classes to be learned may be known in advanced.
- Typically, the learned model is represented in the form of classification rules, decision trees, or mathematical formulae.
- For example, given a database of customer credit information, classification rules can be learned to identify customers having either excellent or fair credit ratings.
- The rules can be used to categorize future data samples, as well as provide a better understanding of the database contents.

Classification Process (1): Model Construction

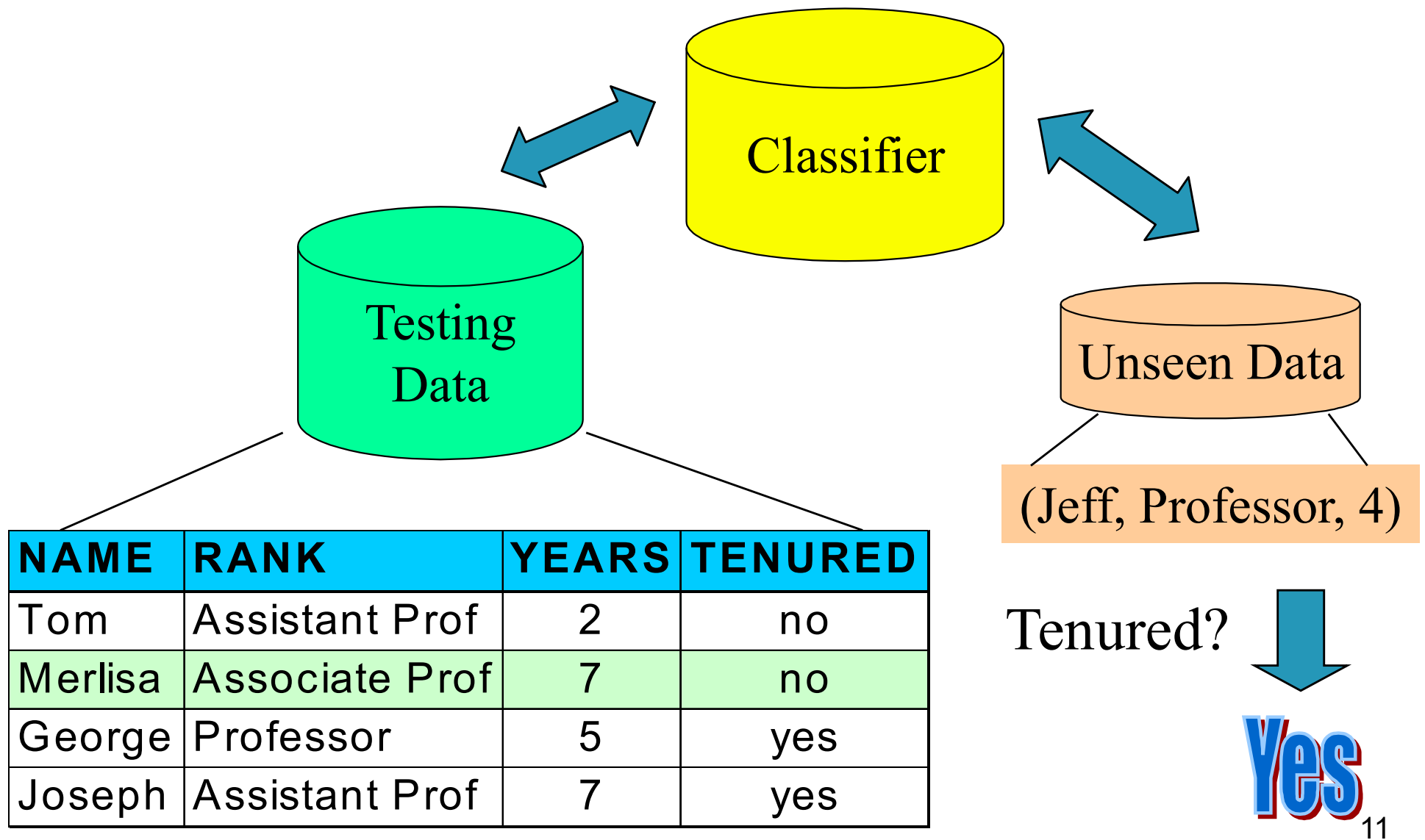


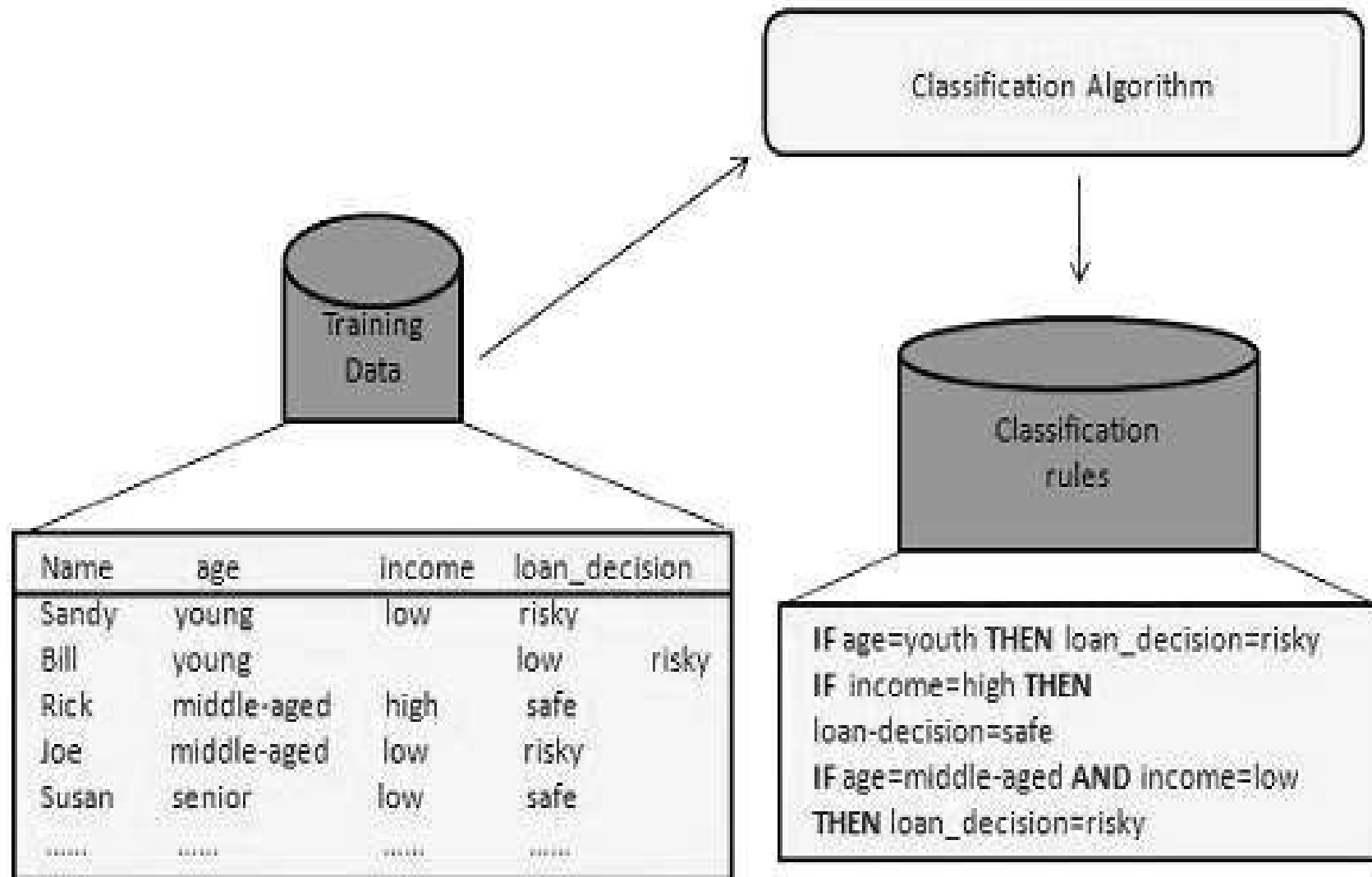
- **Supervised learning (classification)**
 - Supervision: The training data (observations, measurements, etc.) are accompanied by labels indicating the class of the observations
 - New data is classified based on the training set
- **Unsupervised learning (clustering)**
 - The class labels of training data is unknown
 - Given a set of measurements, observations, etc. with the aim of establishing the existence of classes or clusters in the data

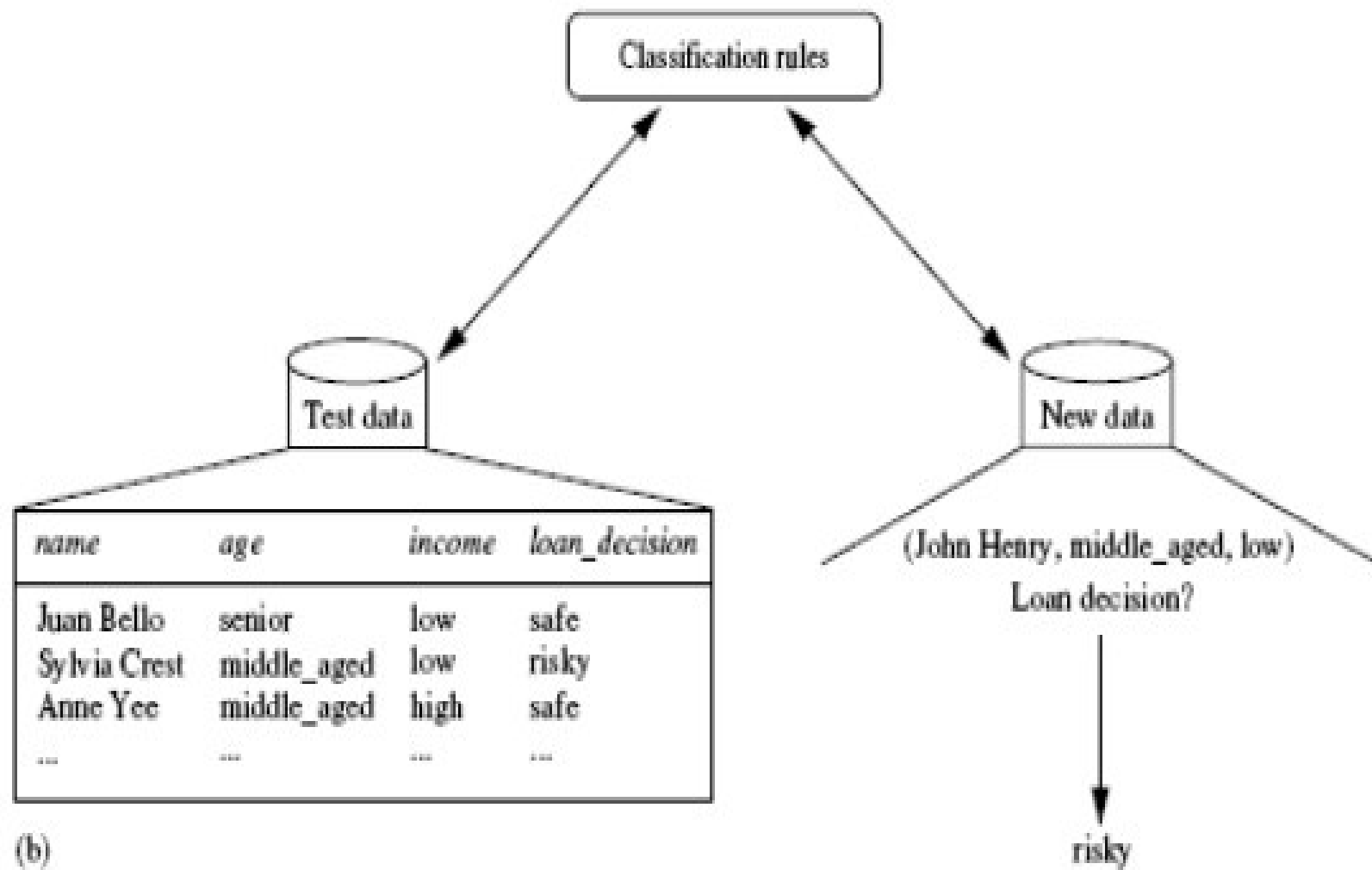
- In the second step, the model is used for classification.
- First the predictive accuracy of the model (or classifier) is estimated. Coming slides we will discuss several methods for estimating classifier accuracy.
- The **Holdout Method** is a simple technique that uses a test set of class labeled samples.
- These samples are randomly selected and are independent of training samples.
- The Accuracy of a model on a given test set is the percentage of test set samples that are correctly classified by the model.
- For each test sample, the known class label is compared with the learned model's class prediction for that sample.
- If the accuracy of model is considered acceptable, the model can be used to classify future data tuples or objects for which the class label is known.
- (Such data are also referred to in the machine learning literature as “unknown” or “previously unseen” data)

- For example, the classification rules learned in Fig a from the analysis of data from existing customers can be used to predict the credit rating of new or future (i.e. previously unseen) customers.
- *“How is predication different from classification”* Predication can be viewed as the construction and use of a model to assess the class of an unlabeled sample, or to assess the value or value ranges of an attribute that a given sample is likely to have

Classification Process (2): Use the Model in Prediction







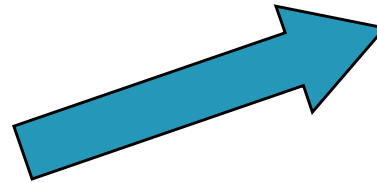
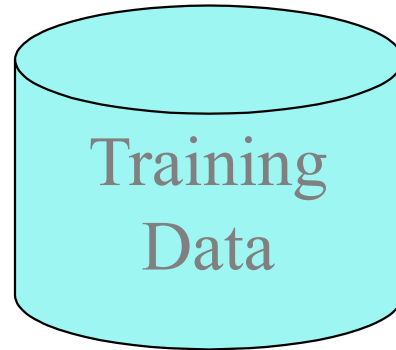
Classification vs. Prediction

- **Classification:**
 - predicts categorical class labels
 - classifies data (constructs a model) based on the training set and the values (**class labels**) in a classifying attribute and uses it in classifying new data
- **Prediction:**
 - models continuous-valued functions, i.e., predicts unknown or missing values
- **Typical Applications**
 - credit approval
 - target marketing
 - medical diagnosis
 - treatment effectiveness analysis

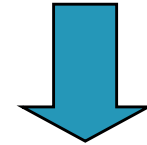
Classification— A Two-Step Process

- **Model construction: describing a set of predetermined classes**
 - Each tuple/sample is assumed to belong to a predefined class, as determined by the **class label attribute**
 - The set of tuples used for model construction: **training set**
 - The model is represented as classification rules, decision trees, or mathematical formulae
- **Model usage: for classifying future or unknown objects**
 - Estimate accuracy of the model
 - The known label of test sample is compared with the classified result from the model
 - Accuracy rate is the percentage of test set samples that are correctly classified by the model
 - Test set is independent of training set, otherwise over-fitting will occur

Classification Process (1): Model Construction



Classification Algorithms

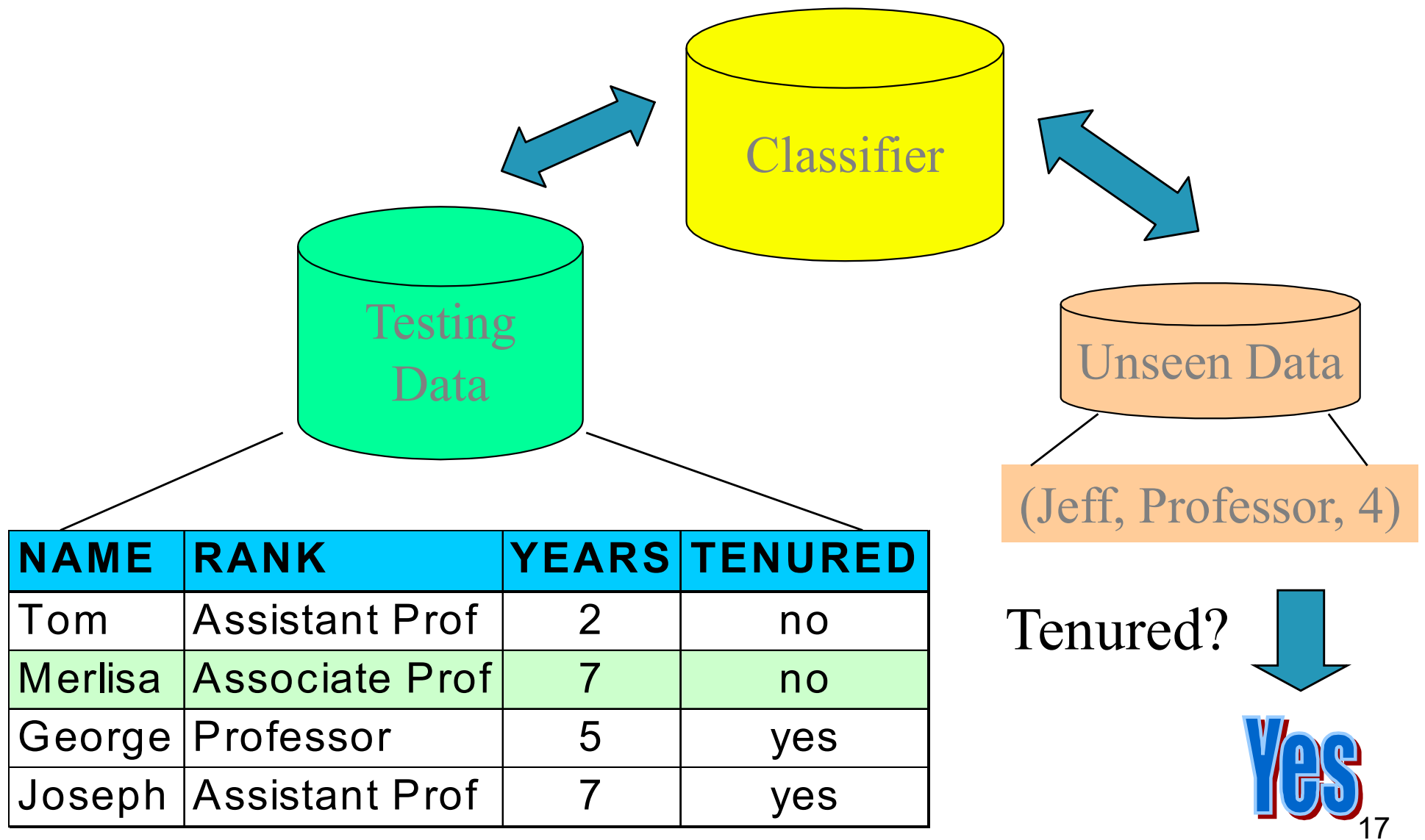


Classifier (Model)

NAME	RANK	YEARS	TENURED
Mike	Assistant Prof	3	no
Mary	Assistant Prof	7	yes
Bill	Professor	2	yes
Jim	Associate Prof	7	yes
Dave	Assistant Prof	6	no
Anne	Associate Prof	3	no

IF rank = 'professor'
OR years > 6
THEN tenured = 'yes'

Classification Process (2): Use the Model in Prediction



Supervised vs. Unsupervised Learning

- **Supervised learning (classification)**
 - Supervision: The training data (observations, measurements, etc.) are accompanied by labels indicating the class of the observations
 - New data is classified based on the training set
- **Unsupervised learning (clustering)**
 - The class labels of training data is unknown
 - Given a set of measurements, observations, etc. with the aim of establishing the existence of classes or clusters in the data

Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian Classification
- Classification by backpropagation
- Classification based on concepts from association rule mining
- Other Classification Methods-KNN
- Prediction
- Classifier Accuracy
- Summary

Issues regarding classification and prediction

Preparing the Data for Classification and Prediction

- Data cleaning
 - This refers to the preprocessing of data in order to remove or reduce noise (by applying smoothing techniques, for ex) and the treatment of missing values (e.g. by replacing the missing values with the most commonly occurring value for the most probable value based on statistics). Although most classification algorithms have some mechanisms for handling noisy or missing data, this step can help reduce confusion during learning
- Relevance Analysis
 - Many of the attributes in the data may be *irrelevant* to the classification or prediction task. For ex, data recording the day of the week on which a bank loan application was filed is unlikely to be relevant to the success of the application. Furthermore, other attributes may be redundant. Hence, relevance analysis may be performed on the data with the aim to removing any irrelevant or redundant attributes from the learning process. In machine learning, this step is known as *feature selection*. Including such attributes may other wise slow down, and possibly mislead, the learning step

- **Data Transformation and reduction:**
 - The data can be generalized to higher – level concepts. Concept hierarchies may be used for this purpose. This is particularly useful for continuous-valued attributes. For ex, numeric values for the attribute income may be generalized to discrete ranges such as low, medium and high. Similarly, nominal-valued attributes, like street, can be generalized to higher level concepts, like city. Since generalization compresses the original training data, fewer input/output operations may be involved during learning.
 - The data may be normalized, particularly when neural networks or methods involving distance measurements are used in the learning step. Normalization involves scaling all values for a given attribute so that they fall within a small specified range, such as -1.0 to 1.0 or 0.0 to 1.0. In methods that use distance measurements, for ex, this would prevent attributes with initially large ranges (like, say income) from outweighing attributes with initially smaller ranges Such as binary attributes.

Comparing Classification Methods

Classification and prediction methods can be compared and evaluated according to the following criteria.

- **Predictive Accuracy** : This refers to the ability of the model to correctly predict the class label of new or previously unseen data
- **Speed** : This refers to the computation costs involved in generating and using the model
- **Robustness** : This is the ability of the model to make correct predictions given noisy data or data with missing values.
- **Scalability** : This refers to the ability to construct the classifier or predictor efficiently given large amounts of data.
- **Interpretability** : This refers to the level of understanding and insight that is provided by the classifier or predictor.

Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian Classification
- Classification by back propagation
- Classification based on concepts from association rule mining
- Other Classification Methods-KNN
- Prediction
- Classifier Accuracy
- Summary

Classification by Decision Tree Induction

- Decision tree
 - A flow-chart-like tree structure
 - Internal node denotes a test on an attribute
 - Branch represents an outcome of the test
 - Leaf nodes represent class labels or class distribution
- Decision tree generation consists of two phases
 - Tree construction
 - At start, all the training examples are at the root
 - Partition examples recursively based on selected attributes
 - Tree pruning
 - Identify and remove branches that reflect noise or outliers
- Use of decision tree: Classifying an unknown sample
 - Test the attribute values of the sample against the decision tree

Training Dataset

This follows an example from Quinlan's ID3

age	income	student	credit_rating
<=30	high	no	fair
<=30	high	no	excellent
31...40	high	no	fair
>40	medium	no	fair
>40	low	yes	fair
>40	low	yes	excellent
31...40	low	yes	excellent
<=30	medium	no	fair
<=30	low	yes	fair
>40	medium	yes	fair
<=30	medium	yes	excellent
31...40	medium	no	excellent
31...40	high	yes	fair
>40	medium	no	excellent

Algorithm for Decision Tree Induction

- Basic algorithm (a greedy algorithm)
 - Tree is constructed in a **top-down recursive divide-and-conquer manner**
 - At start, all the training examples are at the root
 - Attributes are categorical (if continuous-valued, they are discretized in advance)
 - Examples are partitioned recursively based on selected attributes
 - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., **information gain**)
- Conditions for stopping partitioning
 - All samples for a given node belong to the same class
 - There are no remaining attributes for further partitioning – **majority voting** is employed for classifying the leaf
 - There are no samples left

Attribute Selection Measure

- **Information gain** (ID3/C4.5)
 - All attributes are assumed to be categorical
 - Can be modified for continuous-valued attributes
- **Gini index** (IBM IntelligentMiner)
 - All attributes are assumed continuous-valued
 - Assume there exist several possible split values for each attribute
 - May need other tools, such as clustering, to get the possible split values
 - Can be modified for categorical attributes

RID	Age	Income	Student	Credit_rating	Class : Buys_computer
1	=30	High	No	Fair	No
2	<=30	High	No	Excellent	No
3	31.40	High	No	Fair	Yes
4	>40	Medium	No	Fair	Yes
5	>40	Low	Yes	Fair	Yes
6	>40	Low	Yes	Excellent	No
7	31.40	Low	Yes	Excellent	Yes
8	<=30	Medium	No	Fair	No
9	<=30	Low	Yes	Fair	Yes
10	>40	Medium	Yes	Fair	Yes
11	<=30	Medium	Yes	Excellent	Yes
12	31.40	Medium	No	Excellent	Yes
13	31.40	High	Yes	Fair	Yes
14	>40	Medium	No	Excellent	No

Information Gain (ID3/C4.5)

- Select the attribute with the highest information gain
- Assume there are two classes, P and N
 - Let the set of examples S contain p elements of class P and n elements of class N
 - The amount of information, needed to decide if an arbitrary example in S belongs to P or N is defined as

$$I(p, n) = - \frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$

Information Gain in Decision Tree Induction

- Assume that using attribute A a set S will be partitioned into sets $\{S_1, S_2, \dots, S_v\}$
 - If S_i contains p_i examples of P and n_i examples of N , the **entropy**, or the expected information needed to classify objects in all subtrees S_i is

$$E(A) = \sum_{i=1}^v \frac{p_i + n_i}{p + n} I(p_i, n_i)$$

- The encoding information that would be gained by branching on A $Gain(A) = I(p, n) - E(A)$

Attribute Selection by Information Gain Computation

■ Class P:
buys_computer = “yes”

■ Class N:
buys_computer = “no”

■ $I(p, n) = I(9, 5) = 0.940$

■ Compute the entropy for *age*:

age	p_i	n_i	$I(p_i, n_i)$
≤ 30	2	3	0.971
30...40	4	0	0
> 40	3	2	0.971

$$E(\text{age}) = \frac{5}{14} I(2, 3) + \frac{4}{14} I(4, 0) + \frac{5}{14} I(3, 2) = 0.69$$

Hence

$$\text{Gain}(\text{age}) = I(p, n) - E(\text{age})$$

Similarly

$$\text{Gain}(\text{income}) = 0.029$$

$$\text{Gain}(\text{student}) = 0.151$$

$$\text{Gain}(\text{credit_rating}) = 0.048$$

To compute the information gain of each attribute

$$I(S1, S2) = I(9, 5)$$

$$I(p, n) = - \frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$

$$= -9/14 \log 9/14 - 5/14 \log 5/14$$

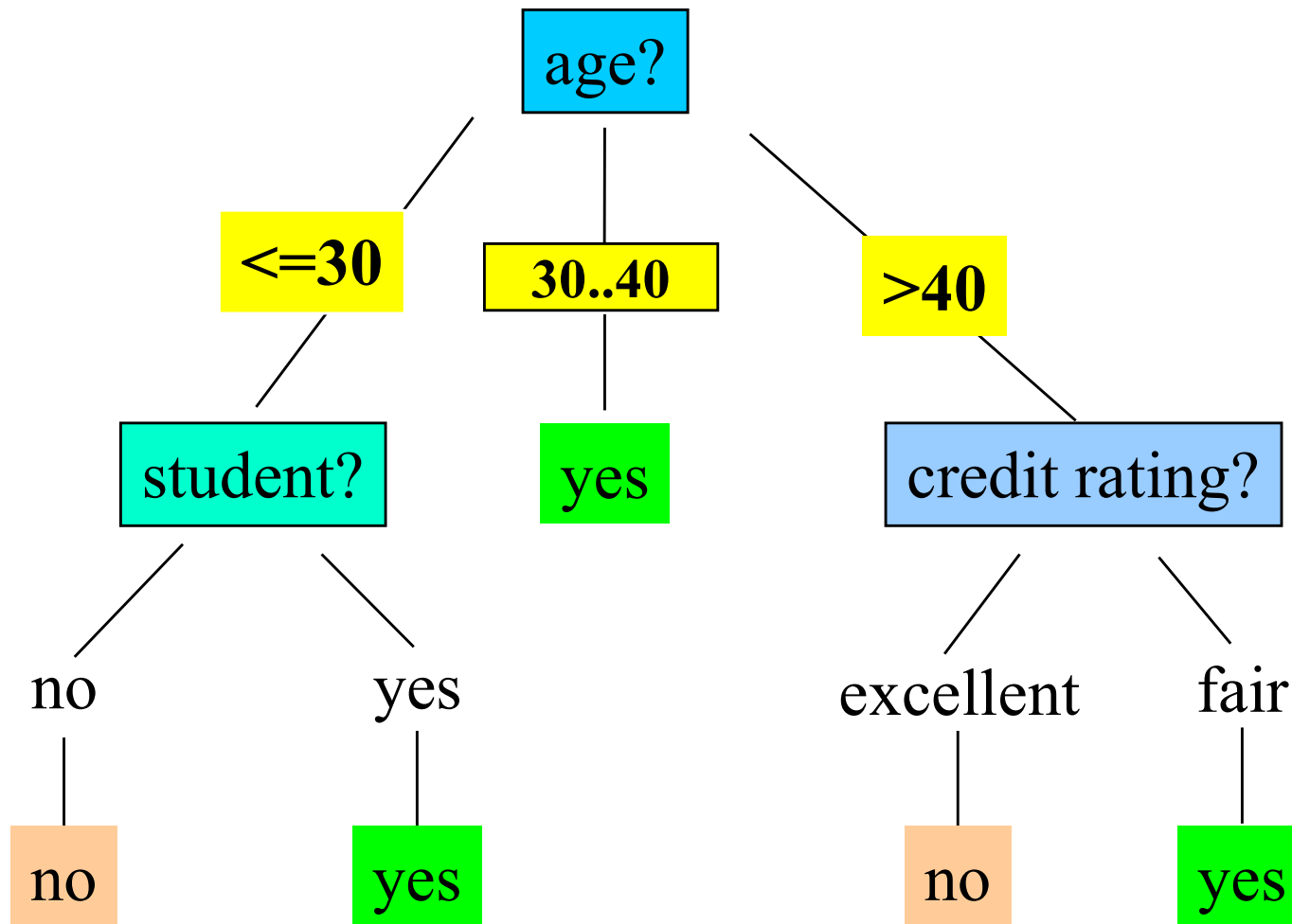
$$= -9/14 \frac{\log_{10}(9/14)}{\log_2(2)} - 5/14 \frac{\log_{10}(5/14)}{\log_2(2)}$$

$$= .6428 * .6374 + .3571 * 1.4854$$

$$= 0.4097 + 0.5304$$

$$= 0.9401$$

Output: A Decision Tree for *“buys_computer”*



Extracting Classification Rules from Trees

- Represent the knowledge in the form of **IF-THEN** rules
- One rule is created for each path from the root to a leaf
- Each attribute-value pair along a path forms a conjunction
- The leaf node holds the class prediction
- Rules are easier for humans to understand
- Example

IF *age* = “≤30” AND *student* = “no” THEN *buys_computer* = “no”

IF *age* = “≤30” AND *student* = “yes” THEN *buys_computer* = “yes”

IF *age* = “31...40” THEN *buys_computer* = “yes”

IF *age* = “>40” AND *credit_rating* = “excellent” THEN
buys_computer = “yes”

IF *age* = “>40” AND *credit_rating* = “fair” THEN *buys_computer* = “no”

Avoid Overfitting in Classification

- The generated tree may overfit the training data
 - Too many branches, some may reflect anomalies due to noise or outliers
 - Result is in poor accuracy for unseen samples
- Two approaches to avoid overfitting
 - Prepruning: Halt tree construction early—do not split a node if this would result in the goodness measure falling below a threshold
 - Difficult to choose an appropriate threshold
 - Postpruning: Remove branches from a “fully grown” tree—get a sequence of progressively pruned trees
 - Use a set of data different from the training data to decide which is the “best pruned tree”

Enhancements to basic decision tree induction

- Allow for continuous-valued attributes
 - Dynamically define new discrete-valued attributes that partition the continuous attribute value into a discrete set of intervals
- Handle missing attribute values
 - Assign the most common value of the attribute
 - Assign probability to each of the possible values
- Attribute construction
 - Create new attributes based on existing ones that are sparsely represented
 - This reduces **fragmentation**
 - **Repetition**- occurs when an attribute is repeatedly tested along a given branch of the tree.
 - **Replication**- duplicate sub trees exist within the tree. It can impede the accuracy and comprehensibility of the resulting tree.

Classification in Large Databases

- Classification—a classical problem extensively studied by statisticians and machine learning researchers
- Scalability: Classifying data sets with millions of examples and hundreds of attributes with reasonable speed
- Why decision tree induction in data mining?
 - relatively faster learning speed (than other classification methods)
 - convertible to simple and easy to understand classification rules
 - can use SQL queries for accessing databases
 - comparable classification accuracy with other methods

Scalable Decision Tree Induction Methods in Data Mining Studies

- **SLIQ** (EDBT'96 — Mehta et al.)
 - builds an index for each attribute and only class list and the current attribute list reside in memory
- **SPRINT** (VLDB'96 — J. Shafer et al.)
 - constructs an attribute list data structure
- **PUBLIC** (VLDB'98 — Rastogi & Shim)
 - integrates tree splitting and tree pruning: stop growing the tree earlier
- **RainForest** (VLDB'98 — Gehrke, Ramakrishnan & Ganti)
 - separates the scalability aspects from the criteria that determine the quality of the tree
 - builds an AVC-list (attribute, value, class label)

Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- **Bayesian Classification**
- Classification by backpropagation
- Classification based on concepts from association rule mining
- Other Classification Methods-KNN
- Prediction
- Classifier Accuracy
- Summary

Bayesian Classification: Why?

- “What are Bayesian Classifiers?” :
- Bayesian Classifiers are statistical classifiers. They can predict class membership probabilities, such as the probability that a given sample belongs to a particular class.
- Bayesian classification is based on Bayes theorem. Studies comparing classification algorithms have found a simple Bayesian classifier known as the naïve Bayesian classifier to be comparable in performance with decision tree and neural network classifier.

Bayesian Classification(contd..)

- Let $\mathbf{X}$ be a data tuple. Where $\mathbf{X}$ is considered “evidence.”
- Let H be some hypothesis, such as that the data tuple $\mathbf{X}$ belongs to a specified class C .
- For classification problems, we want to determine $P(H/\mathbf{X})$, the probability that the hypothesis H holds given the “evidence” or observed data tuple $\mathbf{X}$.
- (we are looking for the probability that tuple $\mathbf{X}$ belongs to class C , given that we know the attribute description of $\mathbf{X}$)
- $P(H/\mathbf{X})$ is the posterior probability, or *a posteriori probability*, of H conditioned on $\mathbf{X}$.

- For example, suppose our world of data tuples is confined to customers described by the attributes *age* and *income*, respectively, and that ***X*** is a 35-year-old customer with an income of \$40,000.
- Suppose that *H* is the hypothesis that our customer will buy a computer.
- $P(H/\mathbf{X})$ reflects the probability that customer ***X*** will buy a computer given that we know the customer's age and income.
- $P(H)$ is the prior probability, or a *priori probability*, of *H*. For our example, this is the probability that any given customer will buy a computer, regardless of age or income.
- $P(\mathbf{X}/H)$ is the posterior probability of ***X*** conditioned on *H*. That is, it is the probability that a customer, ***X***, is 35 years old and earns \$40,000, given that we know the customer will buy a computer.

Bayesian Theorem

- Given training data X , *posteriori probability of a hypothesis H* , $P(H|X)$ follows the Bayes theorem

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}$$

- Practical difficulty: require initial knowledge of many probabilities, significant computational cost

The naïve Bayesian classifier(simple Bayesian classifier) works as follows:

1. Let D be a training set of tuples and their associated class labels. As usual, each tuple is represented by an n -dimensional attribute vector, $\mathbf{X} = (x_1, x_2, \dots, x_n)$, depicting n measurements made on the tuple from n attributes, respectively, $A_1, A_2, \dots, A_n$.
2. Suppose that there are m classes, $C_1, C_2, \dots, C_m$. Given a tuple, $\mathbf{X}$, the classifier will predict that $\mathbf{X}$ belongs to the class having the highest posterior probability, conditioned on $\mathbf{X}$.

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}.$$

3. As $P(\mathbf{X})$ is constant for all classes, only $P(\mathbf{X}/C_i)P(C_i)$ need be maximized. If the class prior probabilities are not known, then it is commonly assumed that the classes are equally likely, that is, $P(C_1) = P(C_2) = \dots = P(C_m)$, and we would therefore maximize $P(\mathbf{X}/C_i)$.
4. Given data sets with many attributes, it would be extremely computationally expensive to compute $P(\mathbf{X}/C_i)$.

$$\begin{aligned} P(\mathbf{X}|C_i) &= \prod_{k=1}^n P(x_k|C_i) \\ &= P(x_1|C_i) \times P(x_2|C_i) \times \dots \times P(x_n|C_i). \end{aligned}$$

5. In order to predict the class label of **X** , $P(\mathbf{X}/C_i)P(C_i)$ is evaluated for each class C_i . The classifier predicts that the class label of tuple **X** is the class C_i if and only if

$$P(X|C_i)P(C_i) > P(X|C_j)P(C_j) \quad \text{for } 1 \leq j \leq m, j \neq i.$$

Naïve Bayesian Classification

- Naïve assumption: attribute independence
- $P(x_1, \dots, x_k | C) = P(x_1 | C) \cdot \dots \cdot P(x_k | C)$
- If i-th attribute is categorical:
 $P(x_i | C)$ is estimated as the relative freq of samples having value x_i as i-th attribute in class C
- If i-th attribute is continuous:
 $P(x_i | C)$ is estimated thru a Gaussian density function
- Computationally easy in both cases

Table 6.1 Class-labeled training tuples from the *AllElectronics* customer database.

<i>RID</i>	<i>age</i>	<i>income</i>	<i>student</i>	<i>credit_rating</i>	<i>Class: buys_computer</i>
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

- **Example:** Predicting a class label using naïve Bayesian classification.
- We wish to predict the class label of a tuple using naïve Bayesian classification. The training data are in Table 6.1.
- The data tuples are described by the attributes *age*, *income*, *student*, and *credit rating*. The class label attribute, *buys computer*, has two distinct values (namely, {*yes*, *no*}).
- Let *C1* correspond to the class *buys computer* = *yes* and *C2* correspond to *buys computer* = *no*.
- The tuple we wish to classify is

$X = (age = youth, income = medium, student = yes, credit\ rating = fair)$

- We need to maximize $P(\mathbf{X}/C_i)P(C_i)$, for $i = 1, 2$. $P(C_i)$, the prior probability of each class, can be computed based on the training tuples:

$$P(\text{buys computer} = \text{yes}) = \frac{9}{14} = 0.643$$

$$P(\text{buys computer} = \text{no}) = \frac{5}{14} = 0.357$$

- To compute $P(\mathbf{X}/C_i)$, for $i = 1, 2$, we compute the following conditional probabilities:

- $P(\text{age} = \text{youth} / \text{buys computer} = \text{yes}) = 2/9 = 0.222$
- $P(\text{age} = \text{youth} / \text{buys computer} = \text{no}) = 3/5 = 0.600$
- $P(\text{income} = \text{medium} / \text{buys computer} = \text{yes}) = 4/9 = 0.444$
- $P(\text{income} = \text{medium} / \text{buys computer} = \text{no}) = 2/5 = 0.400$
- $P(\text{student} = \text{yes} / \text{buys computer} = \text{yes}) = 6/9 = 0.667$
- $P(\text{student} = \text{yes} / \text{buys computer} = \text{no}) = 1/5 = 0.200$
- $P(\text{credit rating} = \text{fair} / \text{buys computer} = \text{yes}) = 6/9 = 0.667$
- $P(\text{credit rating} = \text{fair} / \text{buys computer} = \text{no}) = 2/5 = 0.400$

Using the above probabilities, we obtain

$$P(\mathbf{X}/\text{buys computer} = \text{yes}) =$$

$$\begin{aligned} &P(\text{age} = \text{youth} / \text{buys computer} = \text{yes}) \times \\ &P(\text{income} = \text{medium} / \text{buys computer} = \text{yes}) \times \\ &P(\text{student} = \text{yes} / \text{buys computer} = \text{yes}) \times \\ &P(\text{credit rating} = \text{fair} / \text{buys computer} = \text{yes}) \end{aligned}$$

$$= 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044.$$

Similarly,

$$P(\mathbf{X}/\text{buys computer} = \text{no}) = 0.600 \times 0.400 \times 0.200 \times 0.400 = 0.019.$$

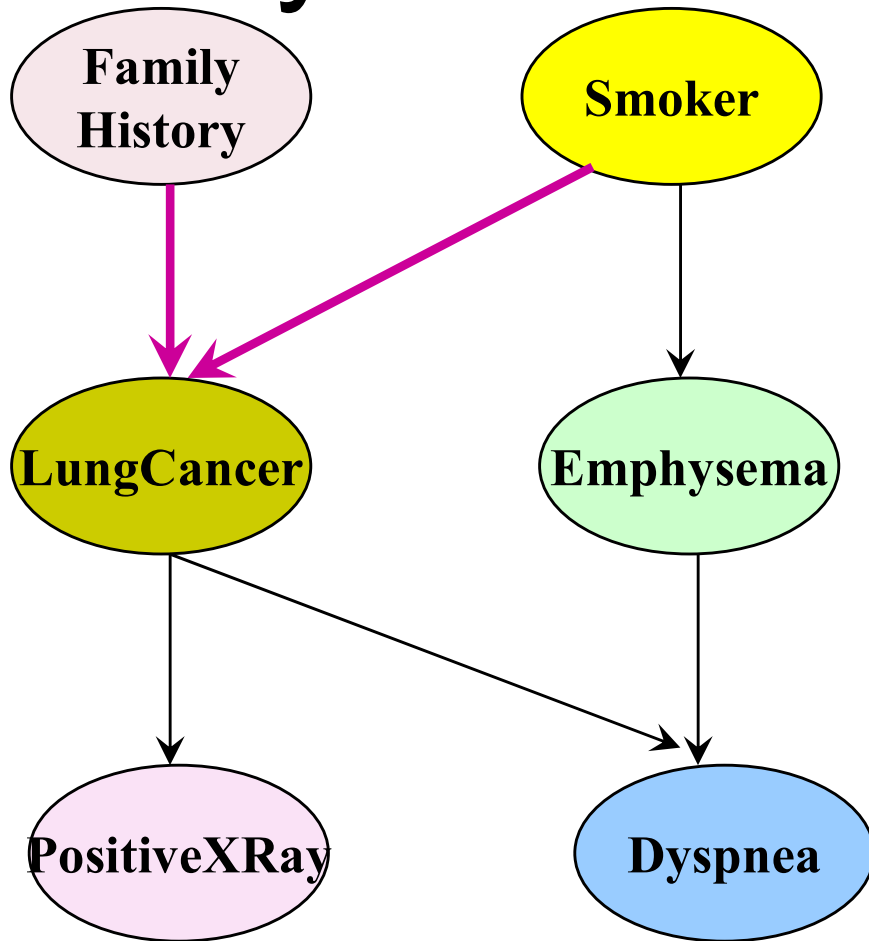
To find the class, C_i , that maximizes $P(\mathbf{X}|C_i)P(C_i)$, we compute

$$\begin{aligned} P(\mathbf{X}/buys_computer = yes)P(buys_computer = yes) \\ = 0.044 \times 0.643 = 0.028 \end{aligned}$$

$$\begin{aligned} P(\mathbf{X}/buys_computer = no)P(buys_computer = no) \\ = 0.0190357 = 0.007 \end{aligned}$$

- **Therefore, the naïve Bayesian classifier predicts *buys_computer = yes* for tuple *X*.**

Bayesian Belief Networks (I)



(FH, S) (FH, ~S) (~FH, S) (~FH, ~S)

LC	0.8	0.5	0.7	0.1
~LC	0.2	0.5	0.3	0.9

**The conditional probability table
for the variable LungCancer**

Bayesian Belief Networks

Bayesian Belief Networks : They provide a graphical model of causal relationships, on which learning can be performed. These networks are also known as Belief Networks.

A Belief Network is defined by two components. The first is a directed acyclic graph, where each node represents a random variable and each arc represents a probabilistic dependence. If an arc is drawn from a node Y to node Z, then Y is a parent or immediate predecessor of Z, and Z is a descendent of Y.

The figure shows a simple belief networks, adapted from the six Boolean Variables.

The arcs allow a representation of causal knowledge. For ex, having lung cancer is influenced by a persons family history of lung cancer, as well as whether or not the person is a smoker. Furthermore, the arcs also show that the variable LungCancer is conditionally independent of Emphysema, given its parents, FamilyHistory and Smoker

This means that once the values of FamilyHistory and Smoker are known, then the variable Emphysema does not provide any additional information regarding LungCancer.

From the conditional probability table (CPT) the upper leftmost and bottom rightmost entries, we see

$$P(\text{LungCancer} = \text{'yes'} \mid \text{FamilyHistory} = \text{'yes'}, \text{Smoker} = \text{'y'}) = 0.8$$

$$P(\text{LungCancer} = \text{'no'} \mid \text{FamilyHistory} = \text{'no'}, \text{Smoker} = \text{'n'}) = 0.9$$

Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian Classification
- Classification by backpropagation
- Classification based on concepts from association rule mining
- Other Classification Methods-KNN
- Prediction
- Classifier Accuracy
- Summary

Classification by Back propagation

What is Back propagation ?

Back propagation is a neural network learning algorithm. The field of neural networks was originally kindled by psychologists, and neurobiologists.

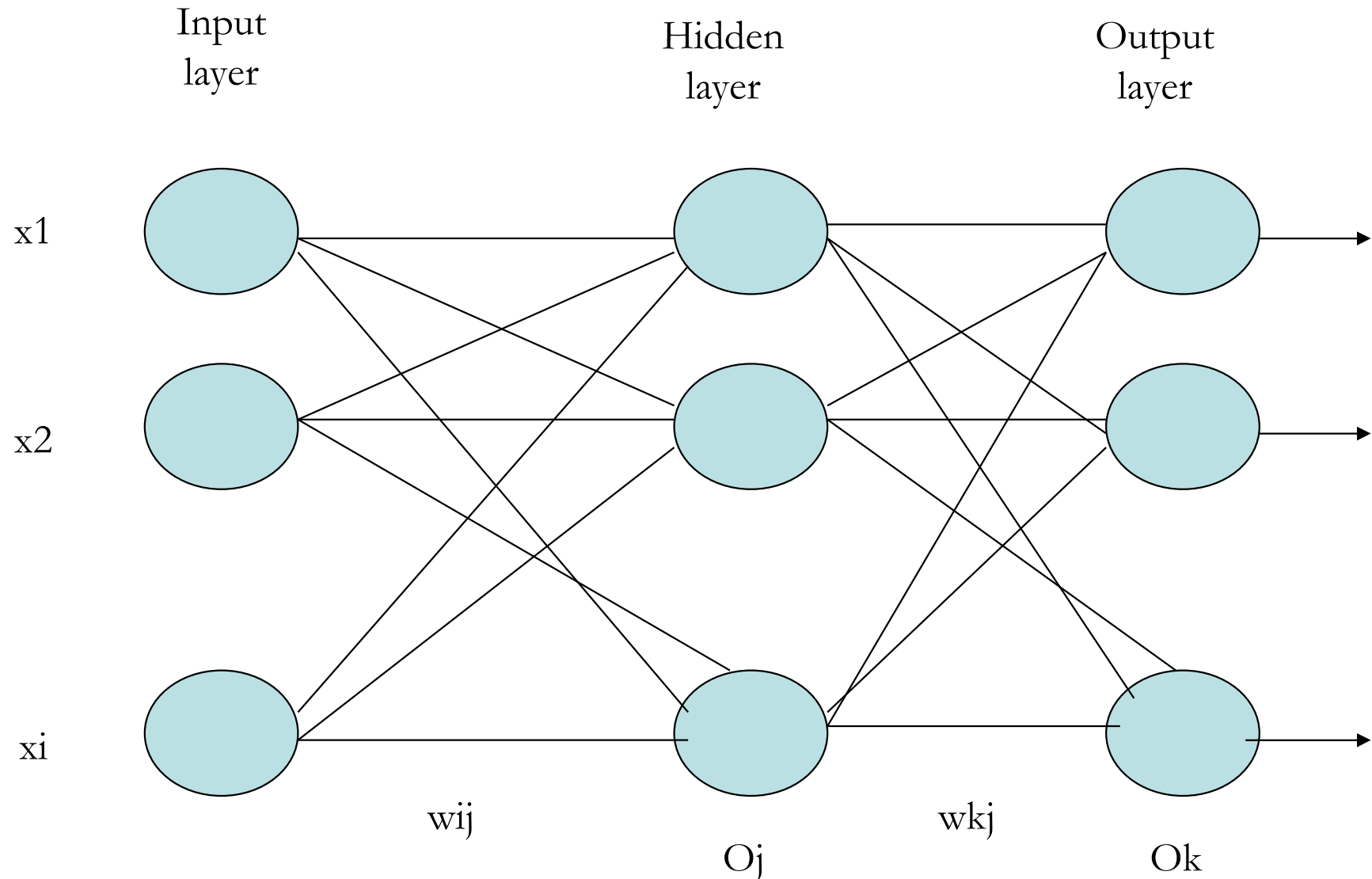
Roughly speaking, a neural network is a set of connected input/output units where each connection has a weight associated with it. During the learning phase, the network learns by adjusting the weights so as to be able to predict the correct class label of the input samples.

Neural Networks

- Advantages
 - prediction accuracy is generally high
 - robust, works when training examples contain errors
 - output may be discrete, real-valued, or a vector of several discrete or real-valued attributes
 - fast evaluation of the learned target function
- Criticism
 - long training time
 - difficult to understand the learned function (weights)
 - not easy to incorporate domain knowledge

- The back propagation algorithm performs learning on a multilayer feed forward neural network. An example of such a network is shown in the figure.
- The inputs correspond to the attributes measured for each training sample. The inputs are fed simultaneously into a layer of units making up the input layer.
- The weighted outputs of these units are, in turn fed simultaneously to a second layer of “neuronlike” units, known as a hidden layer
- The hidden layers weighted outputs can be input to another hidden layer and so on.
- The number of input layers are arbitrary although in practice, usually only one is used.
- The weighted outputs of the last hidden layer are input to units making up to the output layers, which emits the networks prediction for given samples

- The units in the hidden layers and output layer are sometimes referred to as neurodes or output units.

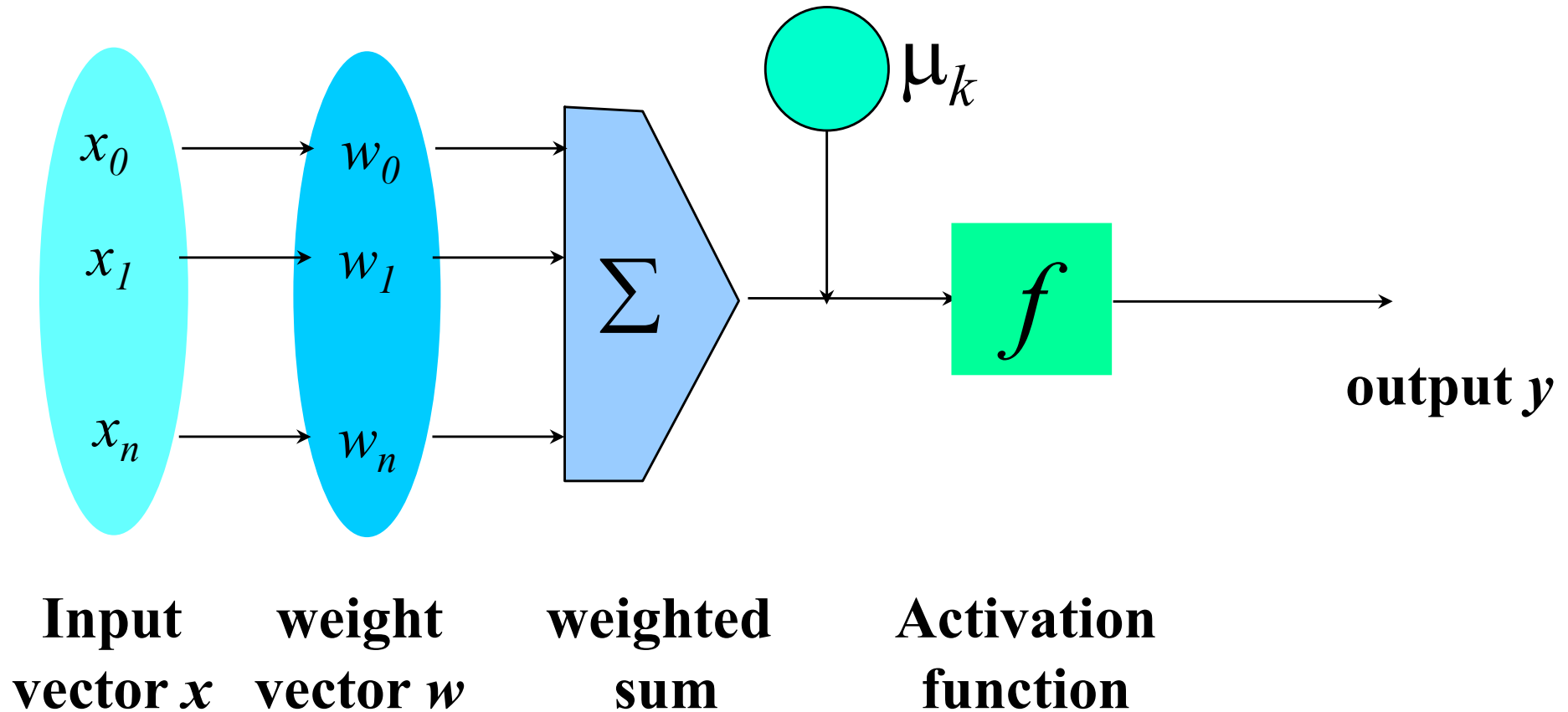


Back propagation

“How does backpropagation work”

- Backpropagation learns by iteratively processing a set of training samples, comparing the networks prediction for each sample with the actual known class label
- For each training sample, the weights are modified so as to minimize the mean squared error between the networks prediction and the actual class.
- These modifications are made in the “backwards” direction, i.e. from output layer, through each hidden down to the first hidden layer (hence the name is back propagation). Although it is not guaranteed, in the general the weights will eventually converge, and the learning process stops.
- Algorithm is given in page 305

A Neuron



- The n -dimensional input vector x is mapped into variable y by means of the scalar product and a nonlinear function mapping

Network Training

- The ultimate objective of training
 - obtain a set of weights that makes almost all the tuples in the training data classified correctly
- Steps
 - Initialize weights with random values
 - Feed the input tuples into the network one by one
 - For each unit
 - Compute the net input to the unit as a linear combination of all the inputs to the unit
 - Compute the output value using the activation function
 - Compute the error
 - Update the weights and the bias

Multi-Layer Perceptron

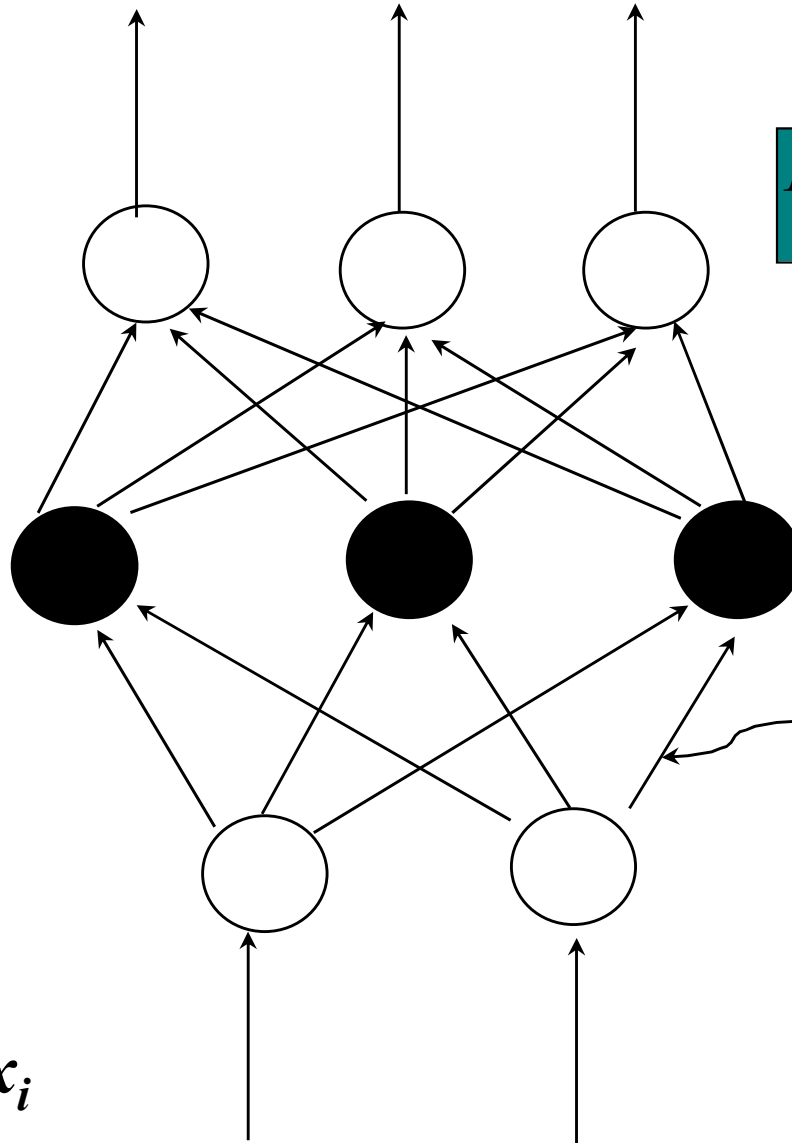
Output vector

Output nodes

Hidden nodes

Input nodes

Input vector: x_i



$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$$

$$\theta_j = \theta_j + (l) Err_j$$

$$w_{ij} = w_{ij} + (l) Err_j O_i$$

$$Err_j = O_j(1 - O_j)(T_j - O_j)$$

$$O_j = \frac{1}{1 + e^{-I_j}}$$

$$I_j = \sum_i w_{ij} O_i + \theta_j$$

Network Pruning and Rule Extraction

- Network pruning
 - Fully connected network will be hard to articulate
 - N input nodes, h hidden nodes and m output nodes lead to $h(m+N)$ weights
 - Pruning: Remove some of the links without affecting classification accuracy of the network
- Extracting rules from a trained network
 - Discretize activation values; replace individual activation value by the cluster average maintaining the network accuracy
 - Enumerate the output from the discretized activation values to find rules between activation value and output
 - Find the relationship between the input and activation value
 - Combine the above two to have rules relating the output to input

Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian Classification
- Classification by backpropagation
- Classification based on concepts from association rule mining
- Other Classification Methods-KNN
- Prediction
- Classifier Accuracy
- Summary

Association-Based Classification

- Several methods for association-based classification
 - ARCS: Quantitative association mining and clustering of association rules (Lent et al'97)
 - It beats C4.5 in (mainly) scalability and also accuracy
 - Associative classification: (Liu et al'98)
 - It mines high support and high confidence rules in the form of “cond_set => y”, where y is a class label
 - CAEP (Classification by aggregating emerging patterns) (Dong et al'99)
 - Emerging patterns (EPs): the itemsets whose support increases significantly from one class to another
 - Mine Eps based on minimum support and growth rate

Mining Quantitative Association Rules

Numeric attributes are *dynamically* discretized during the mining process so as to satisfy some mining criteria, such that the confidence or compactness of the rules mined is maximized.

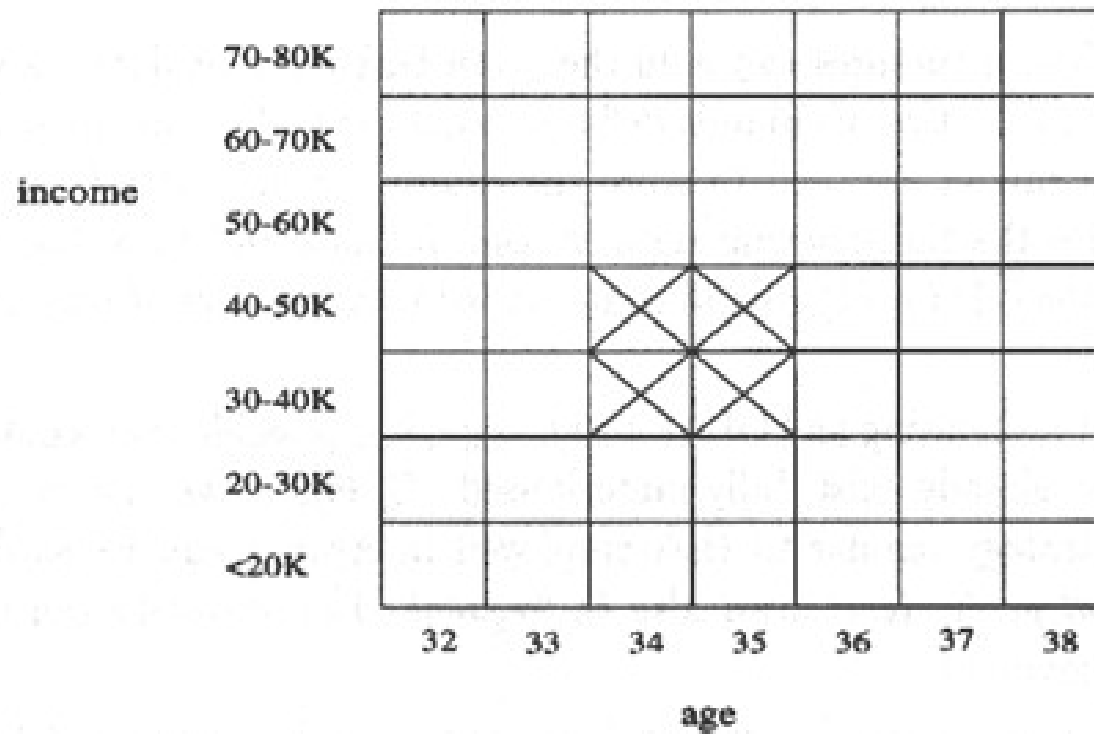
2-D quantitative association rules:

$A_{\text{quan1}} \wedge A_{\text{quan2}} \Rightarrow A_{\text{cat}}$ (two dimensional quantitative association rules).

We can mine such rules by ARCS (association rule clustering system).

Example:

$\text{age}(X, "30-34") \wedge \text{income}(X, "24K - 48K") \Rightarrow \text{buys}(X, "high resolution TV")$

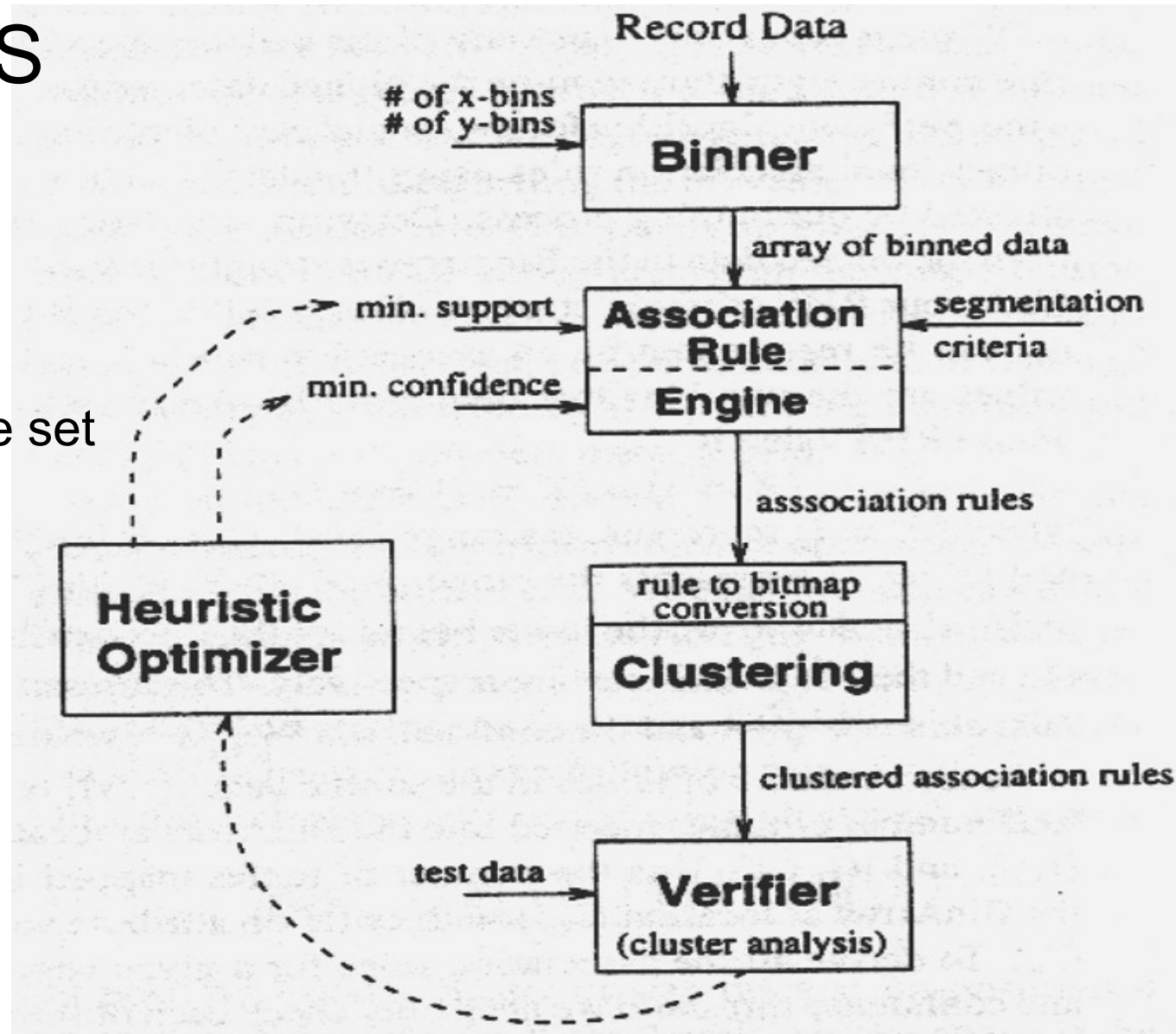


A 2-D grid for tuples representing customers who purchase high resolution TVs

ARCS (Association Rule Clustering System)

How does ARCS work?

1. Binning
2. Find frequent predicate set
3. Clustering
4. Optimize



ARCS

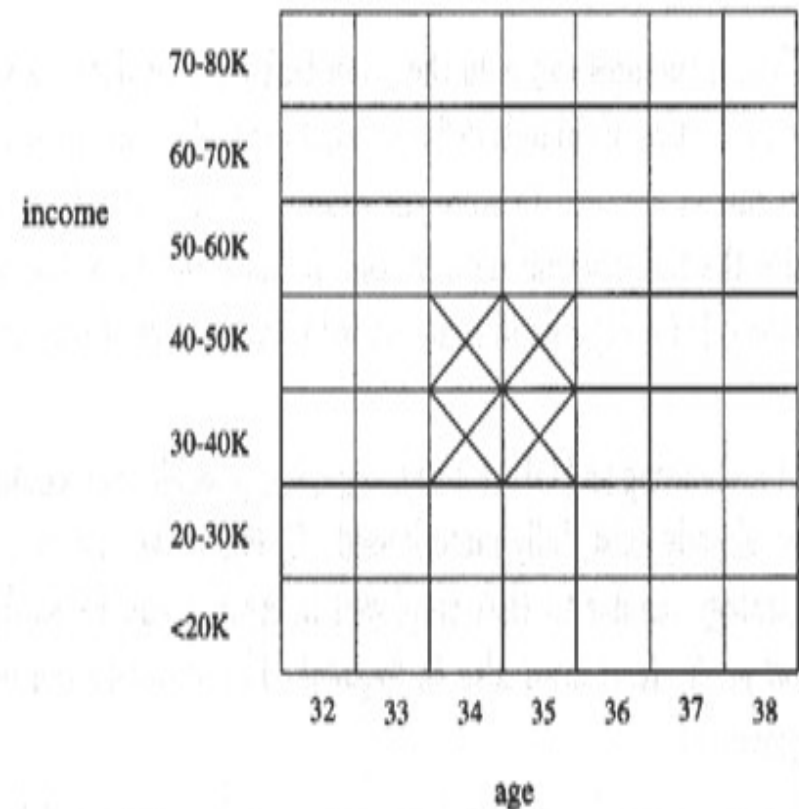
- **Binning:** To keep the grids in a manageable size, we partition the ranges of quantitative attributes into intervals.
- These intervals are dynamic that may further be combined during the mining process.
- This partitioning process is referred to as binning, where intervals are considered as “bins”. Three binning strategies are:
 - **Equiwidth** : where the interval size of each bin is same.
 - **Equidepth** : where each bin has approximately the same no. of tuples assigned to it.
 - **Homogeneity based binning** : where bin size is determined so that the tuples are uniformly distributed in each bin.
 - ARCS uses equi-width binning, where bin size for each quantitative attribute is input by the user.
 - A 2-D array can be used to generate rules for any value of the categorical attribute, based on the two quantitative attributes.

Finding Frequent predicate sets :

- Once 2-D array containing the count distribution for each category is set up, frequent predicate sets are found out (that satisfy min_support) and also which satisfy min_confidence.
- Strong association rules can be generated from these predicate sets using rule generation algo.

Clustering the Association rule:

- Association Rules obtained in the previous step are mapped to a 2-D grid.
- Given the rules,
- **Age(X,34)^income(X,"31k...40k")=>buys(X, "high resolution TV")**
- **Age(X,35)^income(X,"31k...40k")=>buys(X, "high resolution TV")**
- **Age(X,34)^income(X,"41k...50k")=>buys(X, "high resolution TV")**
- **Age(X,35)^income(X,"41k...50k")=>buys(X, "high resolution TV")**
- The above rules can be clustered & replaced
- **Age(X,34..35)^income(X,"31k...50k")=>buys(X, "high resolution TV")**
- ARCS employs a clustering algorithm that scans the grid, searching for rectangular clusters of rules.
- Grid based techniques assumes that the initial association rules can be clustered.
- Prior to clustering, smoothing techniques can be used to remove noise and outliers.



Limitations of ARCS

- Only quantitative attributes on LHS of rules.
- Only 2 attributes on LHS. (2D limitation)
- An alternative to ARCS
 - Non-grid-based
 - equi-depth binning
 - clustering based on a measure of *partial completeness*.

Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian Classification
- Classification by backpropagation
- Classification based on concepts from association rule mining
- Other Classification Methods-KNN
- Prediction
- Classifier Accuracy
- Summary

Other Classification Methods

- k-nearest neighbor classifier
- case-based reasoning
- Genetic algorithm
- Rough set approach
- Fuzzy set approaches

Instance-Based Methods

- Instance-based learning:
 - Store training examples and delay the processing (“lazy evaluation”) until a new instance must be classified
- Typical approaches
 - *k*-nearest neighbor approach
 - Instances represented as points in a Euclidean space.
 - Locally weighted regression
 - Constructs local approximation
 - Case-based reasoning
 - Uses symbolic representations and knowledge-based inference

The k -Nearest Neighbor Algorithm

- The *k-nearest-neighbor method* was first described in the early 1950s. The method is labor intensive when given large training sets, and did not gain popularity until the 1960s
- when increased computing power became available. It has since been widely used in the area of pattern recognition.
- Nearest-neighbor classifiers are based on learning by analogy, that is, by comparing a given test tuple with training tuples that are similar to it. The training tuples are described by n attributes.
- *Each tuple represents a point in an n -dimensional space.*
- *In this way, all of the training tuples are stored in an n -dimensional pattern space.*

- When given an unknown tuple, a ***k-nearest-neighbor classifier searches the pattern space for the k training*** tuples that are closest to the unknown tuple.
- These *k training tuples* are the *k “nearest neighbors”* of the unknown tuple.
- “Closeness” is defined in terms of a distance metric, such as Euclidean distance.
- The Euclidean distance between two points or tuples, say, ***X1 = (x11, x12, : : : , x1n)*** and ***X2 = (x21, x22, : : : , x2n)***, is

$$dist(X_1, X_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}.$$

- For *k*-nearest-neighbor classification, the unknown tuple is assigned the most common class among its *k* nearest neighbors.
- When *k* = 1, the unknown tuple is assigned the class of the training tuple that is closest to it in pattern space.
- Nearest neighbor classifiers can also be used for prediction, that is, to return a real-valued prediction for a given unknown tuple.

Remarks on Lazy vs. Eager Learning

- Instance-based learning: lazy evaluation
- Decision-tree and Bayesian classification: eager evaluation
- Key differences
 - Lazy method may consider query instance x_q when deciding how to generalize beyond the training data D
 - Eager method cannot since they have already chosen global approximation when seeing the query
- Efficiency: Lazy - less time training but more time predicting
- Accuracy
 - Lazy method effectively uses a richer hypothesis space since it uses many local linear functions to form its implicit global approximation to the target function
 - Eager: must commit to a single hypothesis that covers the entire instance space

Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian Classification
- Classification by backpropagation
- Classification based on concepts from association rule mining
- Other Classification Methods-KNN
- Prediction
- Classifier Accuracy
- Summary

What Is Prediction?

- Prediction is similar to classification
 - First, construct a model
 - Second, use model to predict unknown value
 - Major method for prediction is regression
 - Linear and multiple regression
 - Non-linear regression
- Prediction is different from classification
 - Classification refers to predict categorical class label
 - Prediction models continuous-valued functions

Predictive Modeling in Databases

- Predictive modeling: Predict data values or construct generalized linear models based on the database data.
- One can only predict value ranges or category distributions
- Method outline:
 - Minimal generalization
 - Attribute relevance analysis
 - Generalized linear model construction
 - Prediction
- Determine the major factors which influence the prediction
 - Data relevance analysis: uncertainty measurement, entropy analysis, expert judgement, etc.
- Multi-level prediction: drill-down and roll-up analysis

Regress Analysis and Log-Linear Models in Prediction

- Linear regression: $Y = \alpha + \beta X$
 - Two parameters , α and β specify the line and are to be estimated by using the data at hand.
 - using the least squares criterion to the known values of $Y_1, Y_2, \dots, X_1, X_2, \dots$
- Multiple regression: $Y = b_0 + b_1 X_1 + b_2 X_2$.
 - Many nonlinear functions can be transformed into the above.
- Log-linear models:
 - The multi-way table of joint probabilities is approximated by a product of lower-order tables.
 - Probability: $p(a, b, c, d) = \alpha_{ab} \beta_{ac} \chi_{ad} \delta_{bcd}$

Classification and Prediction

- What is classification? What is prediction?
- Issues regarding classification and prediction
- Classification by decision tree induction
- Bayesian Classification
- Classification by backpropagation
- Classification based on concepts from association rule mining
- Other Classification Methods-KNN
- Prediction
- Classifier Accuracy
- Summary

Classifier Accuracy

- Estimating classifier accuracy is important in that it allows one to evaluate how accurately a given classifier will label future data, that is, data on which the classifier has not been trained.
- For example, suppose you used data from previous sales to train a classifier to predict customer purchasing behavior.
- You would like an estimate of how accurately the classifier can predict the purchasing behavior of future customers, that is, future customer data on which the classifier has not been trained.
- Accuracy estimates to help in the comparison of different classifiers.
- We can use these measures in techniques for accuracy estimation, such as the *holdout*, *random subsampling*, *k-fold cross-validation*, and *bootstrap methods*

Estimating Classifier Accuracy

- The holdout method is what we have alluded to so far in our discussions about accuracy.
- In this method, the given data are randomly partitioned into two independent sets, a *training set* and a *test set*. Typically, two-thirds of the data are allocated to the training set, and the remaining one-third is allocated to the test set.
- The training set is used to derive the model, whose accuracy is estimated with the test set
- The estimate is pessimistic because only a portion of the initial data is used to derive the model.
- Random subsampling is a variation of the holdout method in which the holdout method is repeated k times.
- *The overall accuracy estimate is taken as the average of the accuracies obtained from each iteration. (For prediction, we can take the average of the predictor error rates.)*

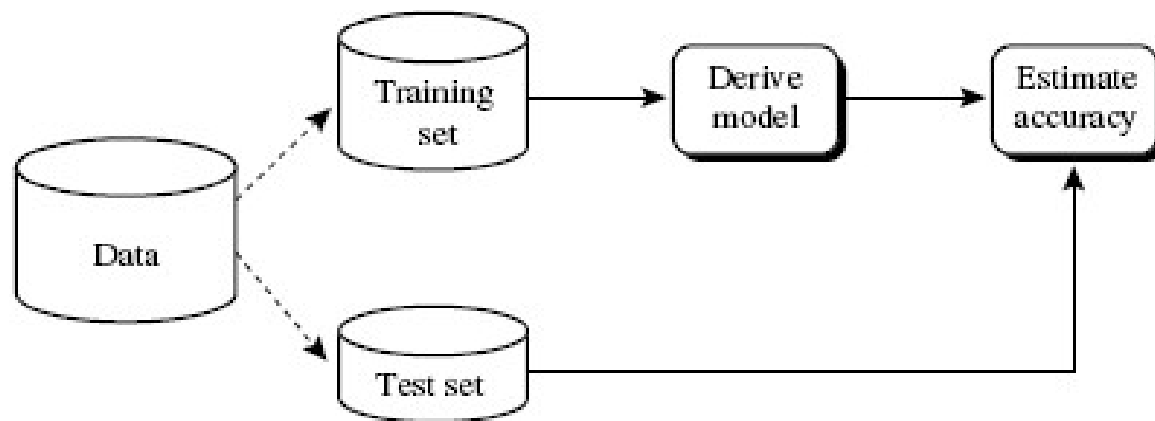
Holdout Method and Random Subsampling

- The holdout method is what we have alluded to so far in our discussions about accuracy.
- In this method, the given data are randomly partitioned into two independent sets, a *training set* and a *test set*.
- *Typically, two-thirds of the data are allocated to the training set, and the remaining one-third is allocated to the test set.*
- The training set is used to derive the model, whose accuracy is estimated with the test set .

The estimate is pessimistic because only a portion of the initial data is used to derive the model.

Random sub sampling is a variation of the holdout method in which the holdout method is repeated *k times*.

The overall accuracy estimate is taken as the average of the accuracies obtained from each iteration. (For prediction, we can take the average of the predictor error rates.)



Estimating accuracy with the holdout method.

Cross-validation

- In ***k-fold cross-validation***, the initial data are randomly partitioned into ***k mutually*** exclusive subsets or “folds,” $D_1, D_2, \dots, D_k$, each of approximately equal size.
- Training and testing is performed *k times*. In iteration *i*, partition D_i is reserved as the test set, and the remaining partitions are collectively used to train the model.
- That is, in the first iteration, subsets $D_2, \dots, D_k$ collectively serve as the training set in order to obtain a first model, which is tested on D_1 ; the second iteration is trained on subsets $D_1, D_3, \dots, D_k$ and tested on D_2 ; and so on. Unlike the holdout and random subsampling methods above, here, each sample is used the same number of times for training and once for testing.
- For classification, the accuracy estimate is the overall number of correct classifications from the *k iterations*, divided by the total number of tuples in the initial data. For prediction, the error estimate can be computed as the total loss from the *k iterations*, divided by the total number of initial tuples.

Bootstrap

- Unlike the accuracy estimation methods mentioned above, the bootstrap method samples the given training tuples uniformly *with replacement*.
- *That is, each time a tuple is selected, it is equally likely to be selected again and readded to the training set.*
- For instance, imagine a machine that randomly selects tuples for our training set. In *sampling with replacement, the machine is allowed to select the same tuple more than once*

Ensemble Methods—Increasing the Accuracy

- Are there *general strategies* for improving classifier and predictor accuracy?
- YES, *Bagging and boosting are two such techniques.*
- We first take an intuitive look at how bagging works as a method of increasing accuracy.
- For ease of explanation, we will assume at first that our model is a classifier. Suppose that you are a patient and would like to have a diagnosis made based on your symptoms.
- Instead of asking one doctor, you may choose to ask several. If a certain diagnosis occurs more than any of the others, you may choose this as the final or best diagnosis.
- That is, the final diagnosis is made based on a majority vote, where each doctor gets an equal vote. Now replace each doctor by a classifier, and you have the basic idea behind bagging.
- Intuitively, a majority vote made by a large group of doctors may be more reliable than a majority vote made by a small group.

- Given a set, D , of d tuples, bagging works as follows.
- For iteration i ($i = 1, 2, \dots, k$), a training set, D_i , of d tuples is sampled with replacement from the original set of tuples, D .
- Note that the term bagging stands for *bootstrap aggregation*.
- A classifier model, M_i , is learned for each training set, D_i .
- To classify an unknown tuple, X , **each classifier, M_i , returns its class prediction, which counts as one vote.**
- **The** bagged classifier, M , counts the votes and assigns the class with the most votes to X .
- Bagging can be applied to the prediction of continuous values by taking the average value of each prediction for a given test tuple.
- The bagged classifier often has significantly greater accuracy than a single classifier derived from D , the original training data.

Boosting

- We now look at the ensemble method of boosting. As in the previous section, suppose that as a patient, you have certain symptoms.
- Instead of consulting one doctor, you choose to consult several. Suppose you assign weights to the value or worth of each doctor's diagnosis, based on the accuracies of previous diagnoses they have made.
- The final diagnosis is then a combination of the weighted diagnoses. This is the essence behind boosting.

- In boosting, weights are assigned to each training tuple.
- A series of k classifiers is iteratively learned. After a classifier M_i is learned, the weights are updated to allow the subsequent classifier, M_{i+1} , to “pay more attention” to the training tuples that were misclassified by M_i .
- The final boosted classifier, M , combines the votes of each individual classifier, where the weight of each classifier’s vote is a function of its accuracy.
- The boosting algorithm can be extended for the prediction of continuous values

Classification Accuracy: Estimating Error Rates

- Partition: Training-and-testing
 - use two independent data sets, e.g., training set (2/3), test set(1/3)
 - used for data set with large number of samples
- Cross-validation
 - divide the data set into k subsamples
 - use $k-1$ subsamples as training data and one subsample as test data --- k -fold cross-validation
 - for data set with moderate size
- Bootstrapping (leave-one-out)
 - for small size data